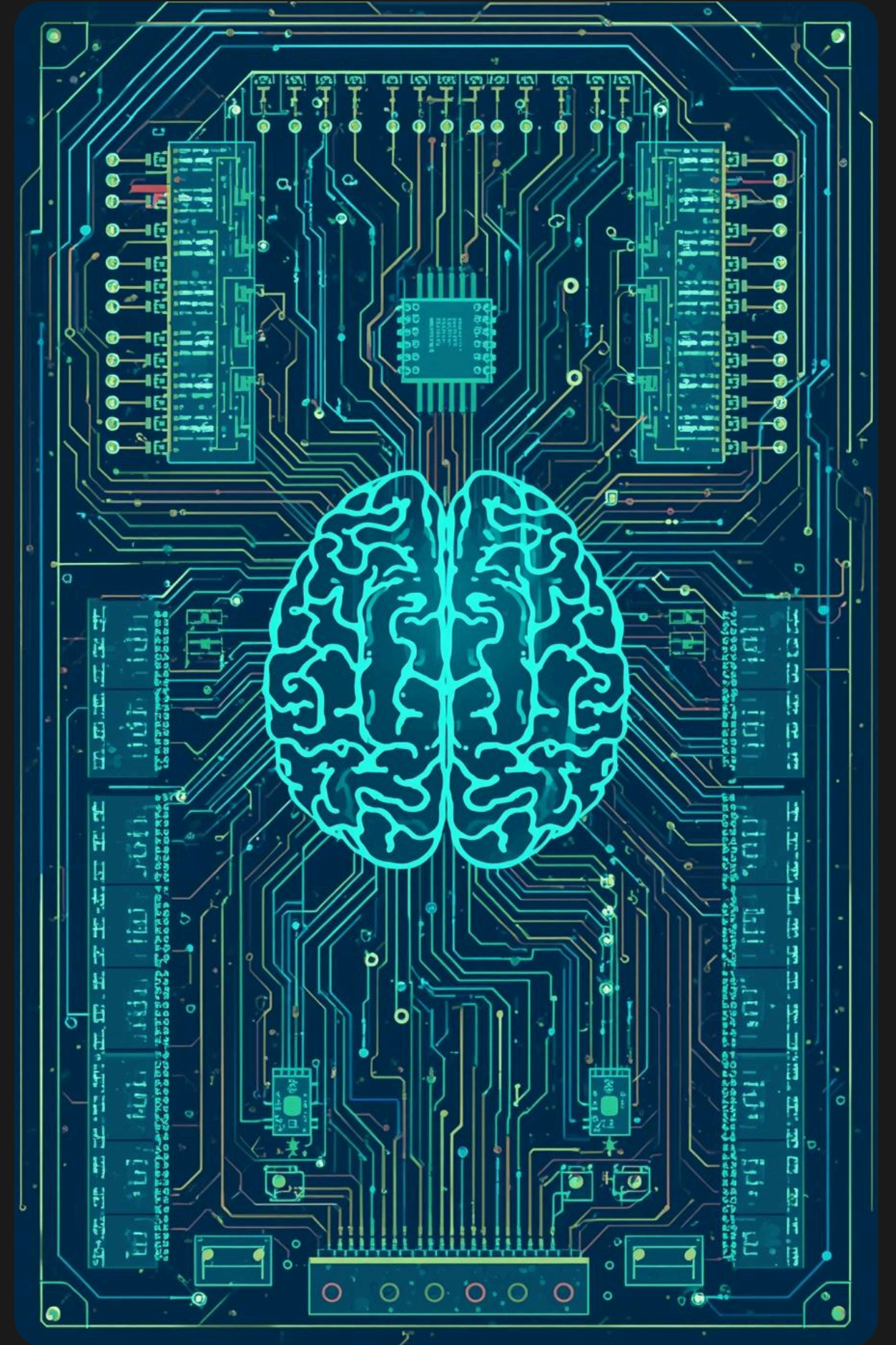


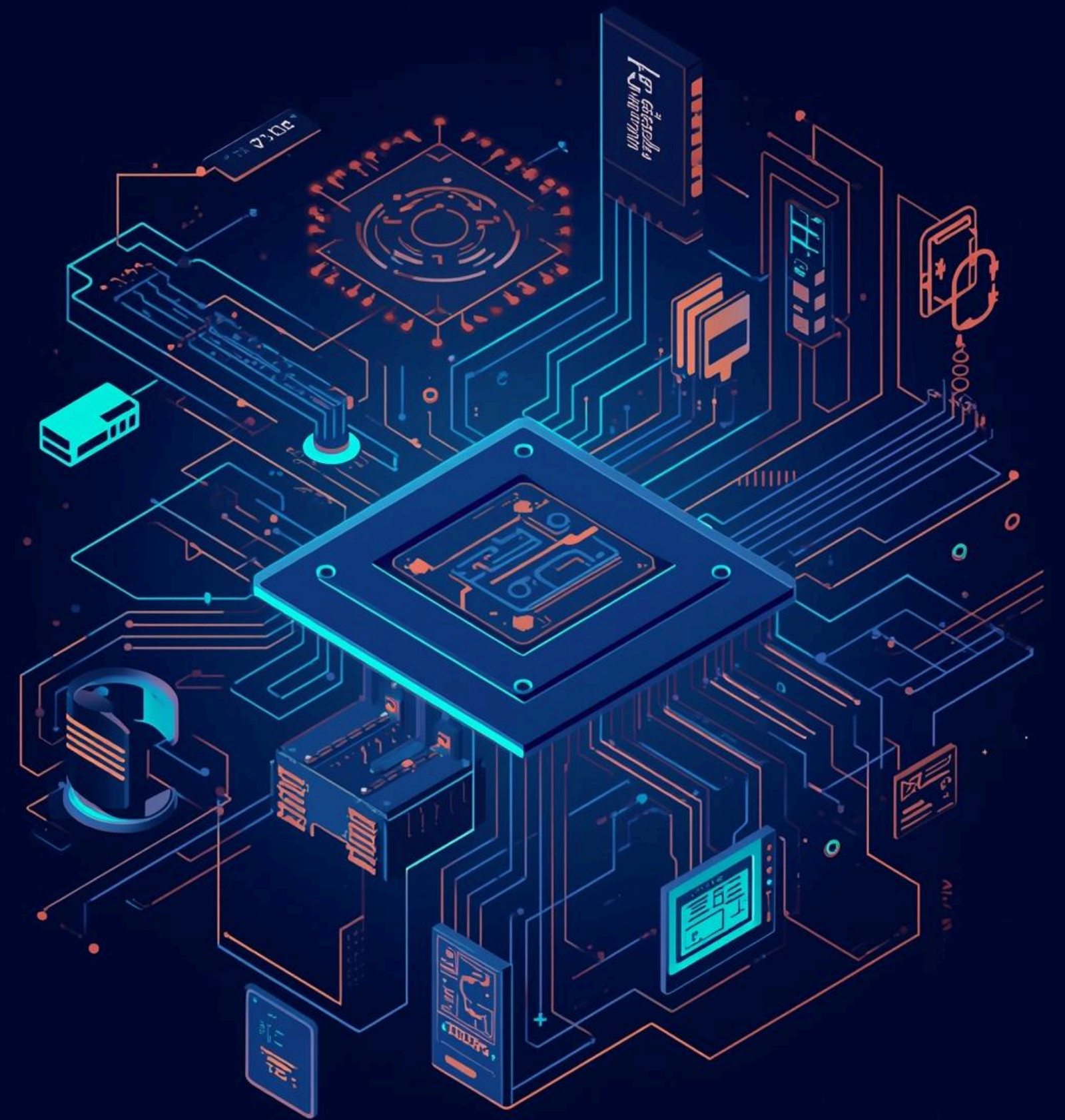
Brain vs. Muscle

Presented by Sushant Niraula



The Hybrid Architecture Explained

The Hybrid Architecture integrates computational power with physical actuation, enabling systems to process information effectively while interacting with the environment. This synergy enhances performance in robotics and embedded systems significantly.



The Raspberry Pi (The Brain)

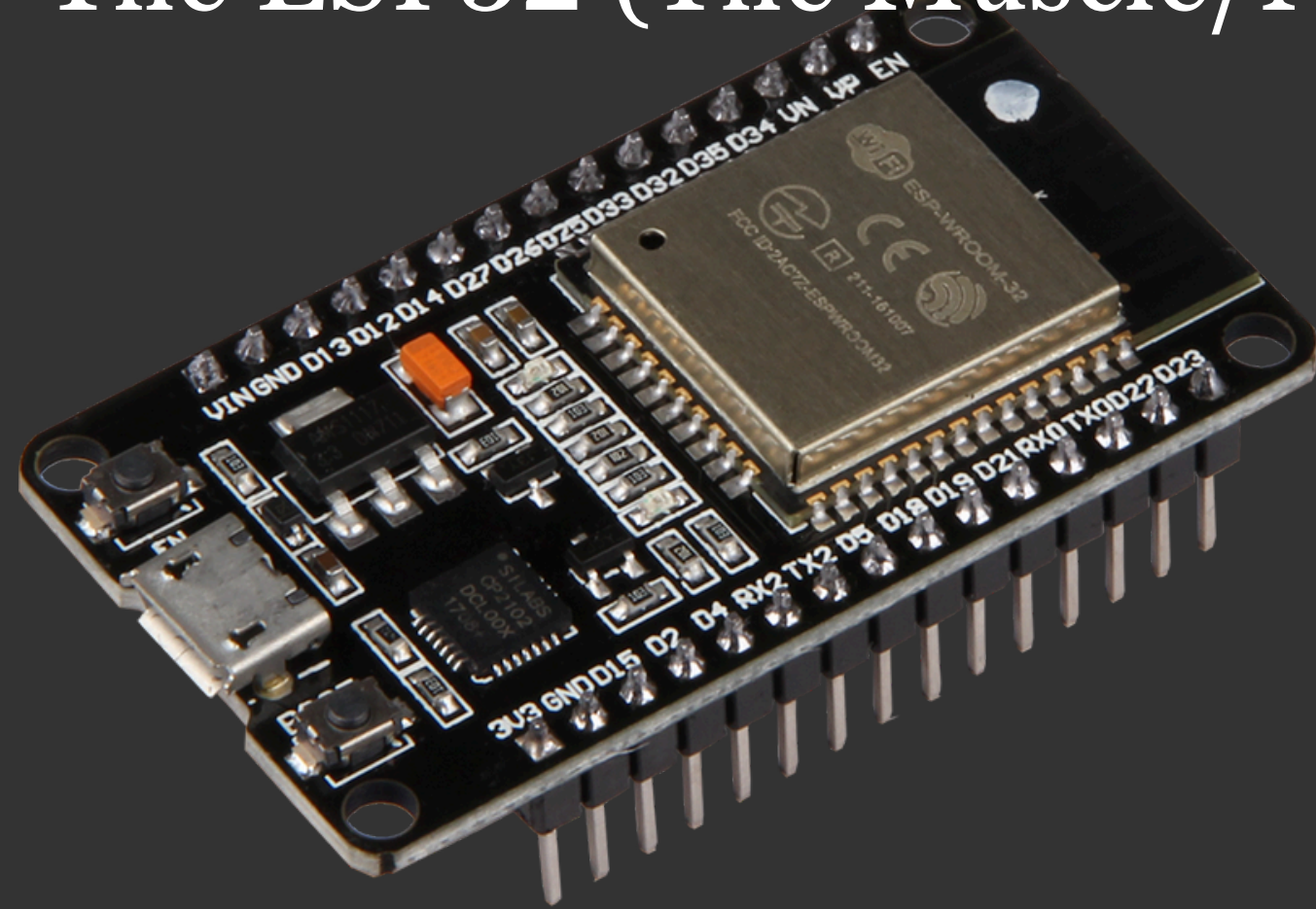


Great at: Python, multitasking, heavy calculations (AI, Databases), internet connectivity.

Limitation: It runs an Operating System (Linux). It is not Real-Time.

Precise timing (like controlling a fast servo or reading a sensitive sensor) can be interrupted by the OS doing background tasks.

The ESP32 (The Muscle/Nerves):



Great at: Real-time control, precise timing (PWM), analog reading.

Limitation: Low processing power compared to Pi, harder to program complex logic.

Low cost compared to PI and so we can make some mess with very minimal consequences.

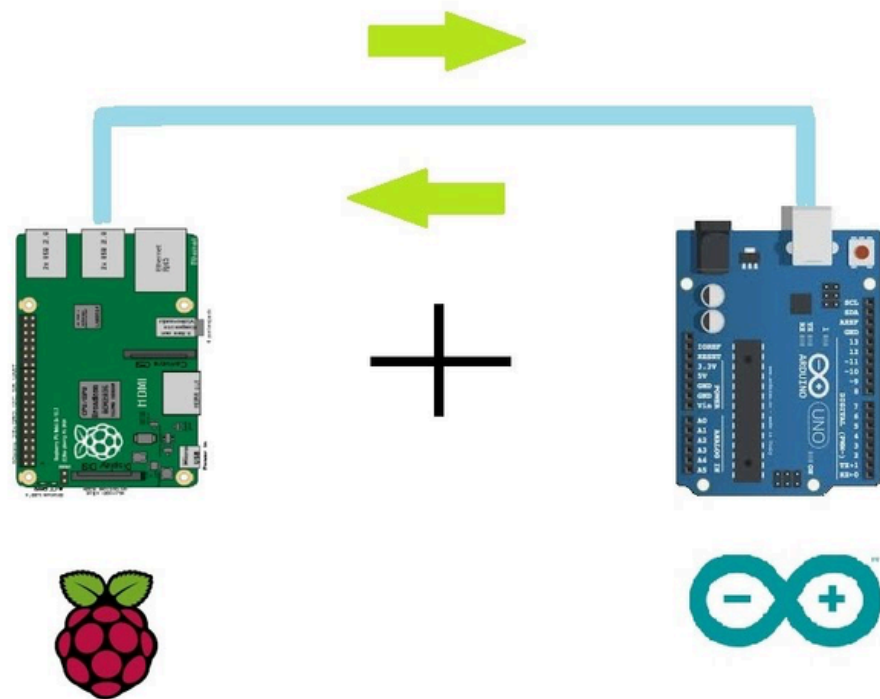
The Hybrid Architecture

We offload the "dirty work" (hardware timing) to the ESP32.

The Pi: Sends high-level commands (e.g., "Set Servo to 90 degrees").

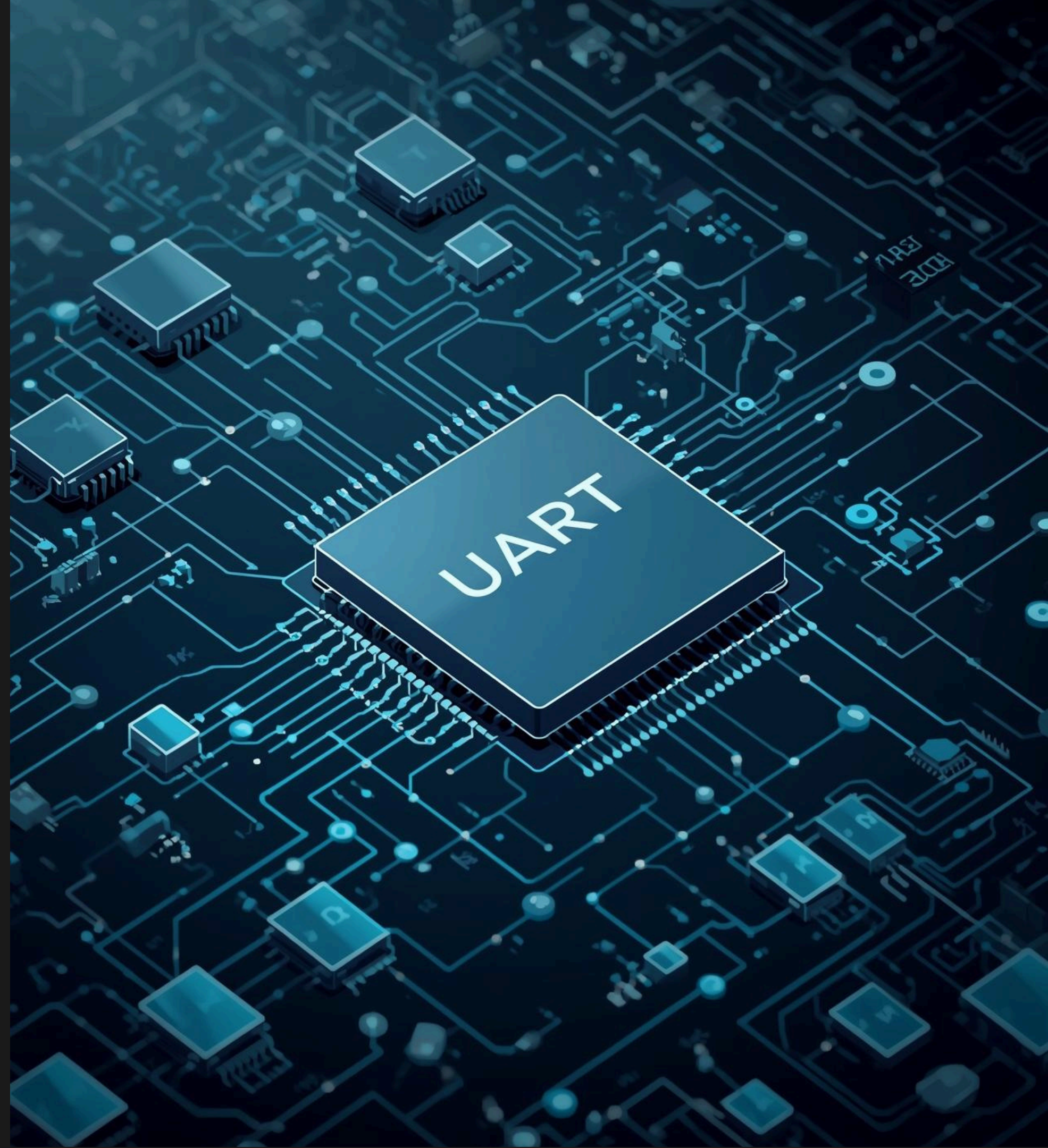
The ESP32: Executes the command and handles the electrical pulses.

The Link: We bridge them using Serial Communication (UART) over the USB cable.



Understanding UART Communication Protocol

The UART protocol enables seamless data transmission between devices by using asynchronous communication. This technique is crucial for controlling actuators and transmitting sensor data in embedded systems applications.



Communication Protocol

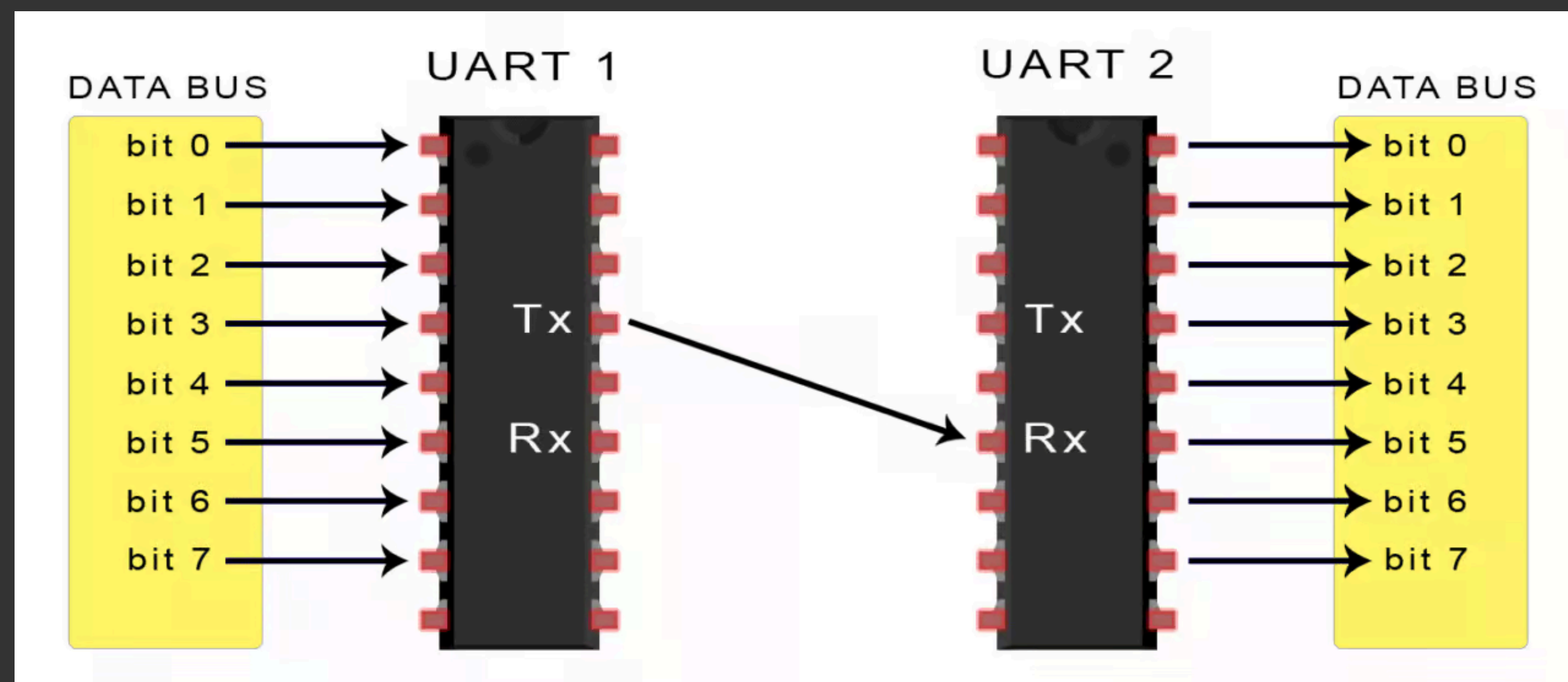
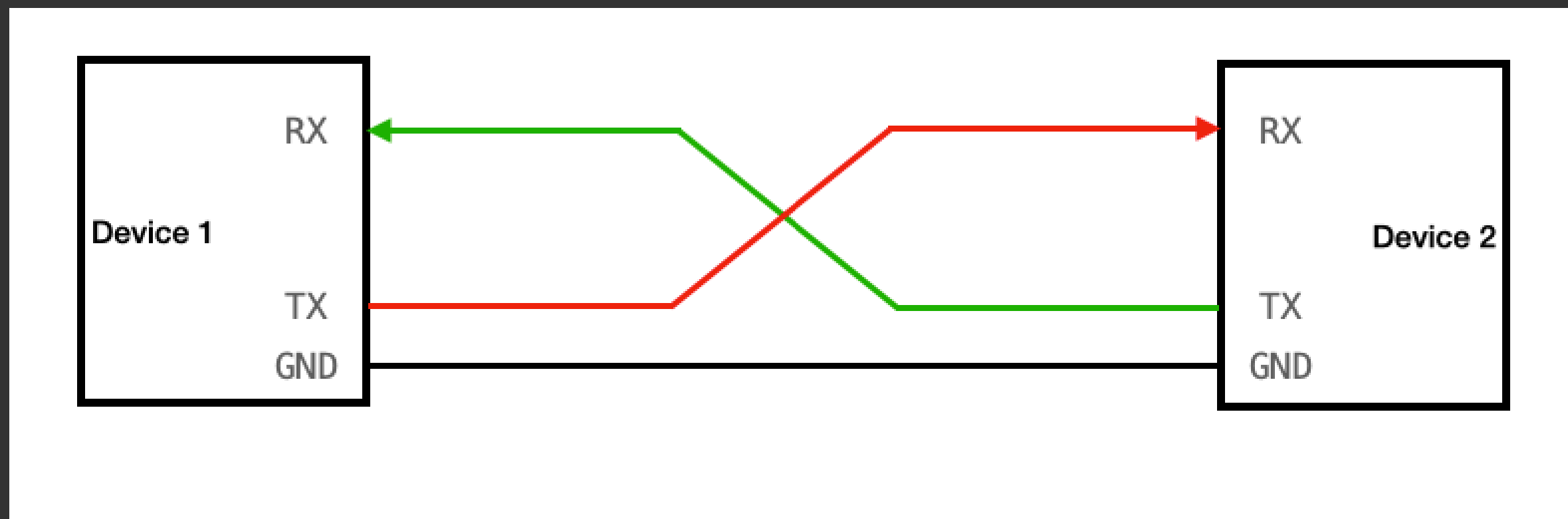
UART (Universal Asynchronous Receiver-Transmitter)

TX (Transmit): The pin that talks.

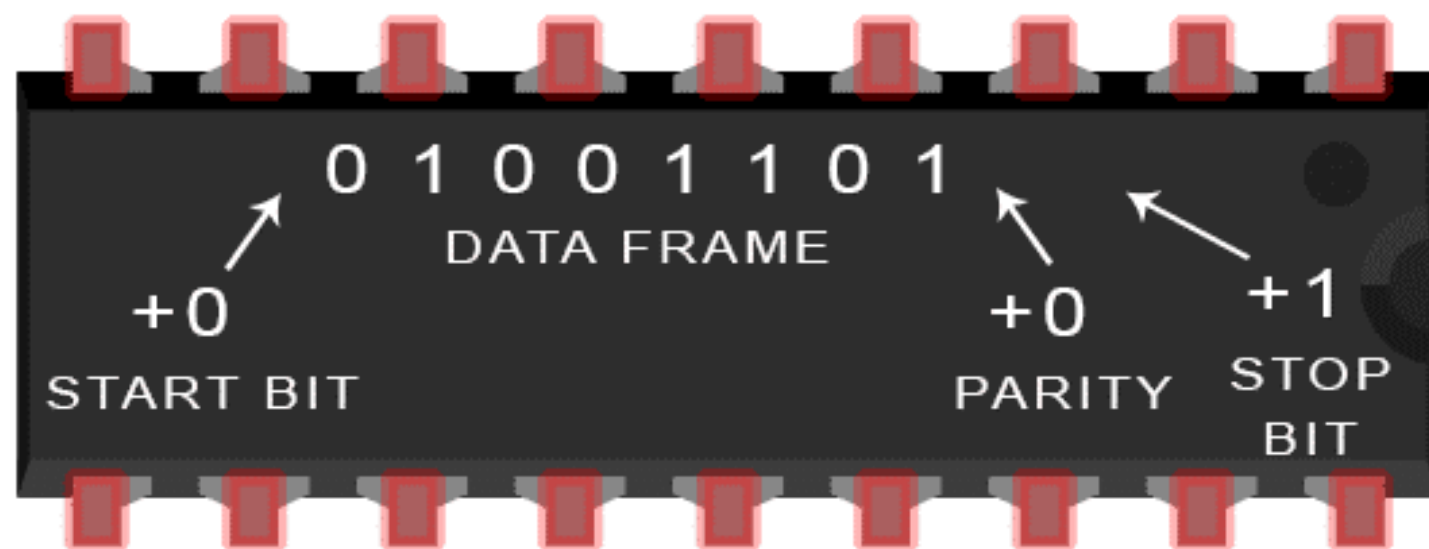
RX (Receive): The pin that listens.

Note: When connected via USB cable, the ESP32's onboard USB-to-UART chip handles this automatically.

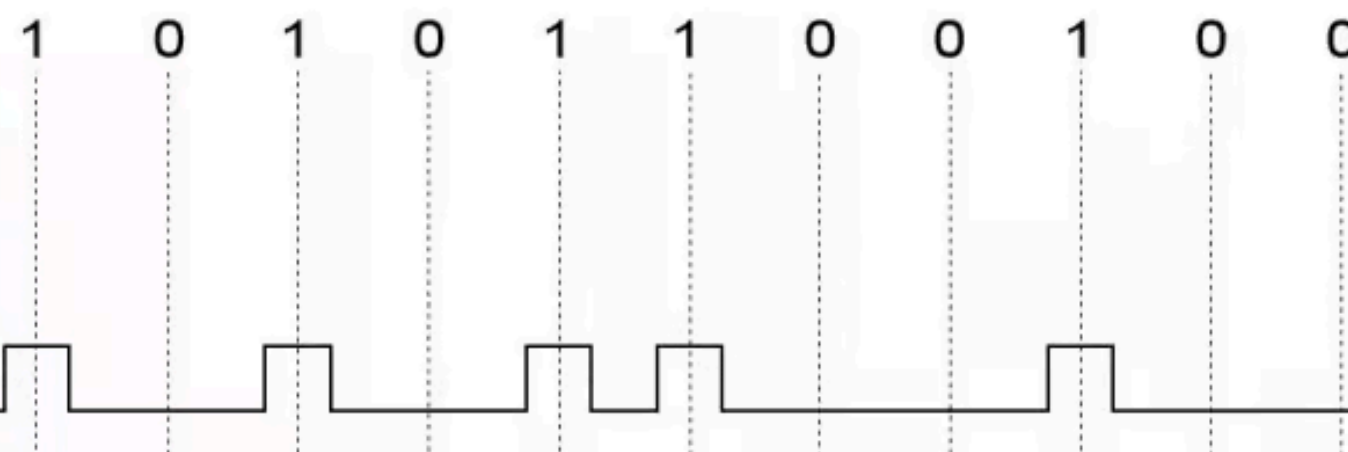
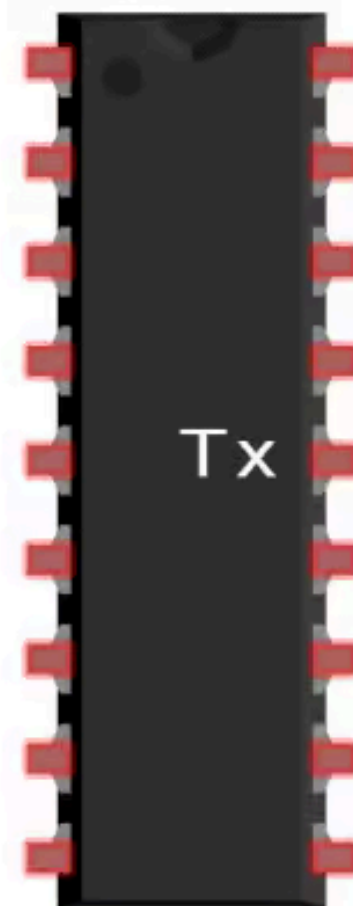
Baud Rate: The speed of talking (e.g., 115200). Must match on both sides!



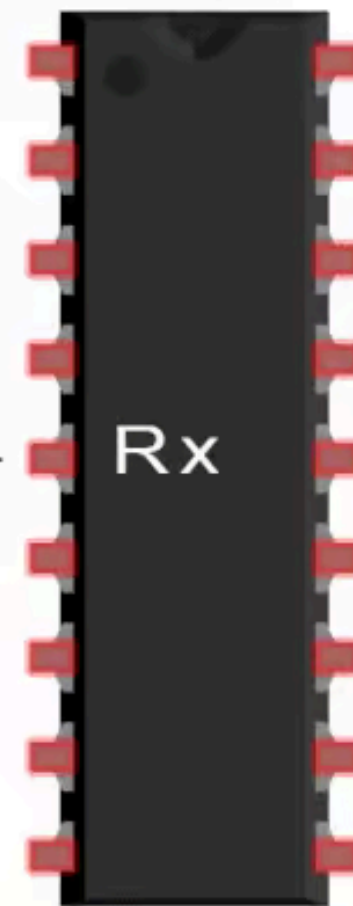
TRANSMITTING UART



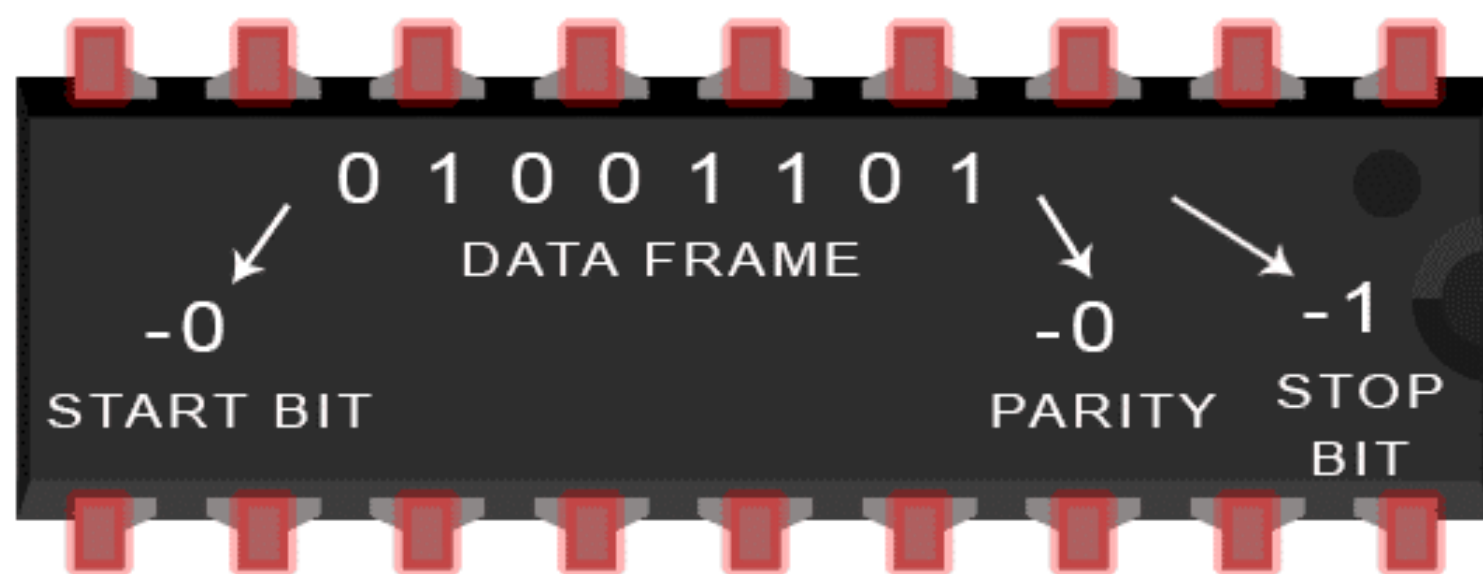
TRANSMITTING UART



RECEIVING UART

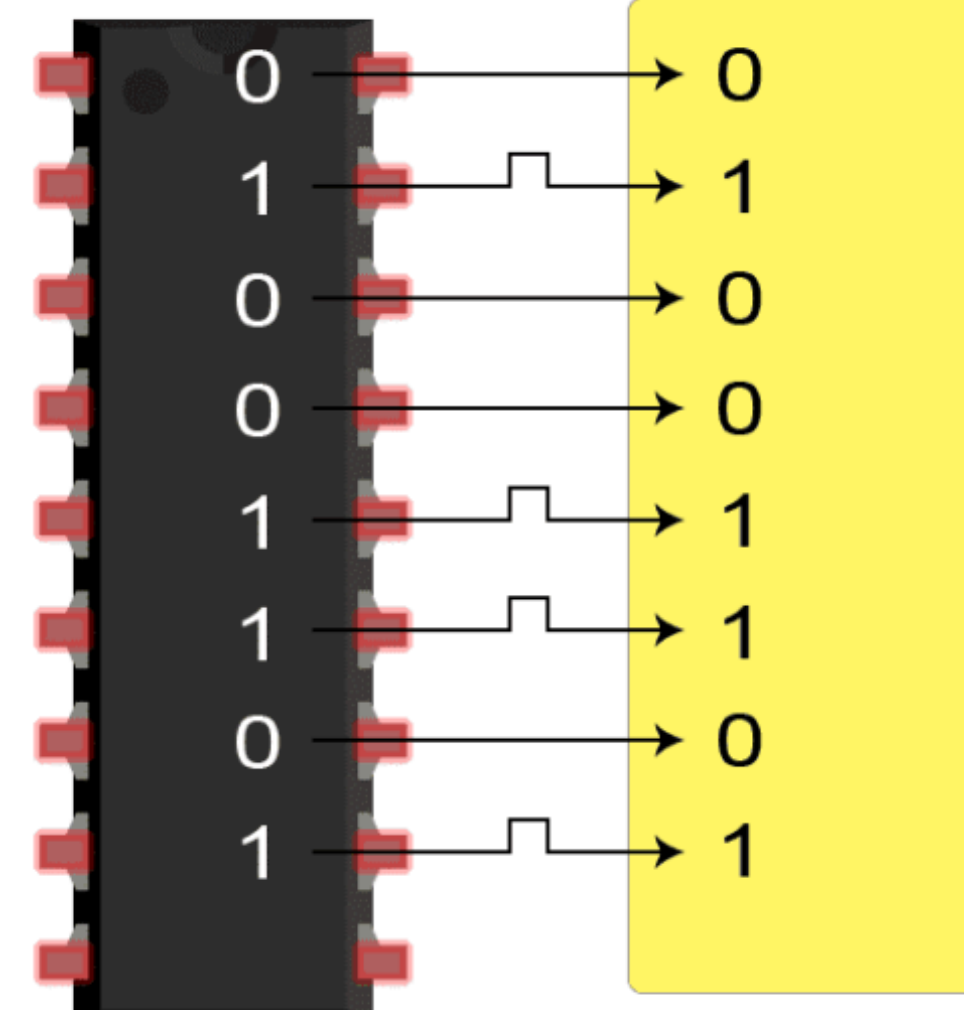


RECEIVING UART



RECEIVING UART

DATA BUS





Next Steps

Continue your journey in electronics!

Lab 1 - Pi Controlling Actuators (Pi → ESP32)

The Raspberry Pi sends a command to toggle an LED . The ESP32 listens and executes.

Hardware Setup:

- ESP32: Connect LED to Pin 2 (Built-in).
- Connection: Connect ESP32 to Raspberry Pi via USB Cable.
- The ESP32 Code (Receiver)


```
1  #include <Arduino.h>
2  const int ledPin = 2;          // Built-in LED on most ESP32s
3
4
5  void setup() {
6      Serial.begin(115200);
7      pinMode(ledPin, OUTPUT);
8  }
9
10 void loop() {
11     if (Serial.available() > 0) {
12         // Read the command string until newline
13         String data = Serial.readStringUntil('\n');
14         data.trim(); // Remove whitespace
15         // Command Parsing
16         // Protocol: "on" = LED ON, "off" = LED OFF
17
18         if (data == "on") {
19             digitalWrite(ledPin, HIGH);
20             Serial.println("ACK: LED ON");
21         }
22         else if (data == "off") {
23             digitalWrite(ledPin, LOW);
24             Serial.println("ACK: LED OFF");
25         }
26         else {
27             Serial.println("NACK: Unknown Command");
28         }
29     }
30 }
```

```
1  import serial
2  import time
3
4  # setup serial
5  try:
6      ser = serial.Serial('/dev/ttyUSB0', 115200, timeout=1.0)
7      time.sleep(3) # Wait for ESP32 reset
8      ser.reset_input_buffer()
9      print("Connected to ESP32. Ready to control!")
10     print("Commands: 'on' (On), 'off' (Off), 'q' (Quit)")
11
12     while True:
13         # Get user command
14         cmd = input("Enter command: ")
15
16         if cmd == 'q':
17             break
18
19         # Send command with a newline character
20         # encode converts string to bytes
21         ser.write((cmd + "\n").encode('utf-8'))
22
23         # Check for acknowledgement from ESP32
24         time.sleep(0.1)
25         while ser.in_waiting > 0:
26             response = ser.readline().decode('utf-8').rstrip()
27             print(f"ESP32 says: {response}")
28
29     except KeyboardInterrupt:
30         print("Exiting...")
31     except serial.SerialException as e:
32         print(f"Error: {e}")
33     finally:
34         if 'ser' in locals() and ser.is_open:
35             ser.close()
```

Lab 2 - ESP32 Sending Sensor Data (ESP32 → Pi)

The ESP32 reads the DHT11 sensor and sends the data to the Pi for processing/logging. This overcomes the Pi's issue with DHT11 timing.

Hardware Setup:

- **ESP32: Connect DHT11 Data pin to Pin 4.**



```
#include "DHT.h"

#define DHTPIN 4
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
}

void loop() {
  // Wait a few seconds between measurements
  delay(2000);

  float h = dht.readHumidity();
  float t = dht.readTemperature();

  // Check if any reads failed
  if (isnan(h) || isnan(t)) {
    Serial.println("ERROR");
    return;
  }

  // Send data in CSV format: "Temperature,Humidity"
  Serial.print(t);
  Serial.print(",");
  Serial.println(h);
}
```

```
import serial
import time
import datetime

ser = serial.Serial('/dev/ttyUSB0', 115200, timeout=1.0)
time.sleep(3)
ser.reset_input_buffer()
print("Logging DHT11 Data...")

try:
    while True:
        if ser.in_waiting > 0:
            try:
                line = ser.readline().decode('utf-8').rstrip()

                if line == "ERROR":
                    print("Sensor Error on ESP32")
                    continue

                # Parse the CSV data
                if "," in line:
                    temp, hum = line.split(",")
                    timestamp = datetime.datetime.now().strftime("%H:%M:%S")

                    print(f"[{timestamp}] Temp: {temp}C | Humidity: {hum}%")

                    # (Optional) You could save this to a file or database here
            except ValueError:
                print("Data packet corrupted")

except KeyboardInterrupt:
    print("Stop logging")
    ser.close()
```



```
#include <Arduino.h>
#include <ESP32Servo.h>
#include "DHT.h"

#define DHTPIN 4
#define DHTTYPE DHT11
#define SERVO_PIN 13

DHT dht(DHTPIN, DHTTYPE);
Servo myServo;

// Timer variables for non-blocking delay
unsigned long previousMillis = 0;
const long interval = 2000; // Read sensor every 2000ms (2 seconds)

void setup() {
  Serial.begin(115200);
  dht.begin();
  myServo.attach(SERVO_PIN);
  myServo.write(0); // Start position (Fan Off)
}

void loop() {
  unsigned long currentMillis = millis();

  // --- TASK 1: Read Sensor (every 2 seconds) ---
  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;

    float h = dht.readHumidity();
    float t = dht.readTemperature();
```

```
    if (!isnan(h) && !isnan(t)) {
        // Send: "T,24.5,H,60"
        Serial.print("T,");
        Serial.print(t);
        Serial.print(",H,");
        Serial.println(h);
    }
}

// --- TASK 2: Listen for Servo Commands (Always running) ---
if (Serial.available() > 0) {
    String command = Serial.readStringUntil('\n');
    command.trim(); // Clean up whitespace

    if (command == "FAN_ON") {
        myServo.write(180); // Move servo to "On" position
    }
    else if (command == "FAN_OFF") {
        myServo.write(0); // Move servo to "Off" position
    }
}
}
```

```
import serial
import time

# --- CONFIGURATION ---
PORT = '/dev/ttyUSB0' # Check your port!
BAUD = 115200
TEMP_THRESHOLD = 28.0 # Temperature to trigger the fan

# Initialize Serial
try:
    ser = serial.Serial(PORT, BAUD, timeout=1.0)
    time.sleep(3) # Wait for ESP32 reset
    ser.reset_input_buffer()
    print(f"System Active. Fan starts above {TEMP_THRESHOLD}C")
except Exception as e:
    print(f"Connection Failed: {e}")
    exit()

# --- MAIN LOOP ---
try:
    while True:
        if ser.in_waiting > 0:
            try:
                # 1. Read and clean the line
                line = ser.readline().decode('utf-8').rstrip()

                # 2. Parse Data (Expects: "T,25.0,H,60.0")
                if line.startswith("T,"):
                    parts = line.split(',')
                    # parts[0]="T", parts[1]="25.0", parts[2]="H", parts[3]="60.0"
```



```

        current_temp = float(parts[1])
        current_hum = float(parts[3])

        print(f"Temp: {current_temp}C | Hum: {current_hum}%", end="")

    # 3. THE BRAIN LOGIC
    if current_temp >= TEMP_THRESHOLD:
        print(" -> Status: HOT! Fan ON")
        ser.write(b"FAN_ON\n")
    else:
        print(" -> Status: Normal. Fan OFF")
        ser.write(b"FAN_OFF\n")

except ValueError:
    print(" | Data Error")
except IndexError:
    print(" | Packet Incomplete")

time.sleep(0.01) # Prevent CPU overload

except KeyboardInterrupt:
    print("\nShutting down system...")
    ser.write(b"FAN_OFF\n") # Safety: Turn off fan before quitting
    ser.close()

```

